



VulnHunterX

— LLM-VERIFIED SAST · HANDS-ON WORKSHOP

Find real bugs. Skip the false alarms.

Static analysis finds candidates — an LLM decides which are real.

65 custom CodeQL queries

108 custom Semgrep rules

8 languages · 5 profiles

60-minute workshop · for developers · CodeQL + Semgrep + LLM triage

Agenda — 4 parts

VulnHunterX

1 · VulnHunterX introduction

Features · architecture · methodology · results

2 · How to install & use

Toolchain, install, configure, run a scan

3 · Real example — dvpwa

Scan a vulnerable Python web app, verify the verdicts

4 · Homework — dvcp

Scan the Damn Vulnerable C Program at home



PART 1

VulnHunterX — features, architecture, methodology, results



From noise to signal

SAST over-approximates by design — the cost is human triage, not the scan.

Hundreds of raw SAST findings

*every program point that *might* be vulnerable*



LLM verification — Vulnhalla guided triage

evidence-anchored, multi-turn reasoning



A handful of real, exploitable bugs

what your team should actually fix

Real bugs get
buried.

Teams stop
reading the
report.

A Python framework that pairs **SAST engines** with **multi-turn LLM verification** to suppress false positives — the **Vulnhalla** guided-question methodology.

Cite evidence

the LLM must point at real code

Pull context

callers, structs, globals on demand

Reason in turns

not pattern-match one snippet

Key features

VulnHunterX

8 languages

C, C++, Python, JavaScript, PHP, Java, Go, C#

3 SAST engines

CodeQL · Semgrep · OpenGrep (--tool ...)

5 rule profiles

standard → extended → maximum → ... → full

LLM providers

OpenAI · Anthropic · Ollama — via LiteLLM

Multi-turn verification

context expansion → verdict + confidence

Flexible inputs

Git URL · local dir · batch repos.yaml

Reports

Markdown EN/VI — summary + per-finding detail

Guided questions

Rule-specific checklists grounded in source evidence

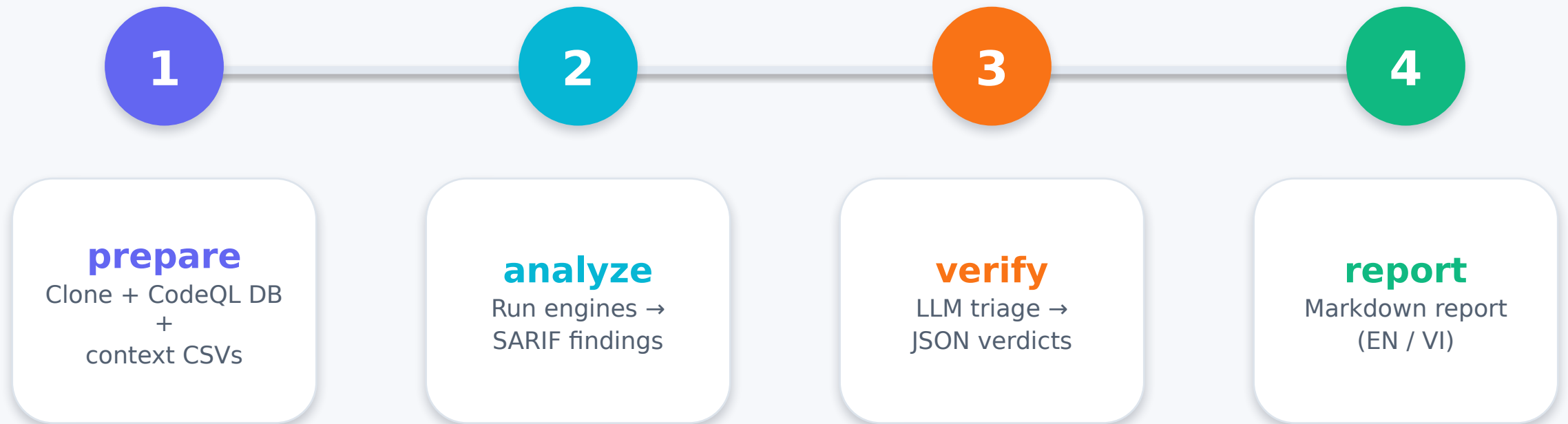
The pipeline — 4 stages, 1 command

VulnHunterX



scan runs prepare → analyze → verify → report

The 4 core stages



The Vulnhalla multi-turn loop

VulnHunterX



Loop up to **max_iterations (default 3)** until the LLM commits.

Guided questions: the per-language contract

```
      # config/prompts/python_questions.yaml – one bank per language
py/sql-injection:
  short_description: User input concatenated into SQL query
  questions:      # answered IN ORDER, before any verdict
    - Quote the EXACT sink (cursor.execute, line N) and name the variable.
    - List EVERY assignment to that variable on each path, with line numbers.
    - Does the value derive from request.args/form/json, or a safe source?
    - Name the SPECIFIC defense – parameterised execute(sql, params), ORM.
  context_hint: Trace request input -> transformations -> query execution.
  additional_context: [caller, callees]  # fetched on demand mid-loop
  min_iterations: 2      # force a second reasoning pass
```

Same CWE, every language — CWE-89 / SQL injection

py/sql-injection

sink: cursor.execute(...) · defense: parameterised execute(sql, params) / ORM .filter()

java/sql-injection

sink: Statement.executeQuery(...) · defense: PreparedStatement with bound params

js/sql-injection

sink: db.query(`...\${x}`) · defense: parameterised driver call / placeholders

php/sql-injection

sink: mysqli_query(...) · defense: prepared statements / PDO bound params

Same ordered checklist shape — language-specific sinks, sanitizers & framework defenses.

8 banks cpp · cs · go · java · js · php · python (+ default fallback)

No SQL bank for C/C++ — *instead C/C++ gets memory-safety banks: use-after-free, integer-overflow.*

Routing a finding to its question bank

QuestionsLoader resolves every finding to a bank through ordered fallback tiers.

exact: ruleId matches a bank key — CodeQL: py/sql-injection

normalized · prefix · lang_prefix: tolerate id variants & same-language partials

cwe: Semgrep has no CodeQL-style id → route by CWE tag via cwe_question_map
(config/rule_categories.yaml): CWE-89 → "sql-injection" → py/sql-injection

default → generic: fall back to default_questions.yaml, then an 8-step generic checklist

Multi-iteration override taint CWEs (22/78/79/89/90/94/611/918...) + access-control (200/264/287/862/863...) force min_iterations = 2. **OWASP-python CWE-22: 57.1% (1 iter) → 95.8% (2 iter).** *Taint gated to framework langs; access-control all langs.*

What a verdict looks like

Finding #12 · cpp/use-after-free

TRUE
POSITIVE

Confidence

High · 0.86

Reasoning

Tainted length from recv() reaches memcpy() with no bounds check; attacker-controlled. Confirmed by fetching caller parse_packet(), which frees buf before the copy. Reachable on the error path.

2 turns

CWE-416

verdict TP / FP / Needs More Data

confidence High / Medium / Low + score

reasoning evidence-anchored, cites code

iterations turns the LLM took

data_flow source→sink trace (CodeQL)

Rules & coverage

VulnHunterX

Counted from config/ at build time — not copied from the README.

65

custom CodeQL queries

cpp 16 · csharp 5 · go 8 · java 10 ·
javascript 14 · python 12

108

custom Semgrep rules

cpp 4 · csharp 14 · go 19 · java 16 ·
javascript 14 · php 14 · python 27

~365

**guided-question sets · 8
banks**

rule-specific checklists

5 rule profiles standard → extended → maximum → extended-registry → full
124-entry CWE routing map routes findings to the right guided questions
~5-10× more rules from --profile standard to --profile full

Result — false positives cut by 91%

VulnHunterX

OWASP-Python benchmark · DeepSeek · 300 findings · raw SAST → VulnHunterX.

87.3%

Precision

up from 37.7% (raw SAST)

92.4%

F1 score

up from 54.7%

91.4%

False positives removed

of all raw-SAST false alarms

Recall held at 98.2% real bugs are kept, not thrown away with the noise

True-positive preservation 97.4% only 1.0% of findings end up 'Needs More Data'

Raw SAST precision was 37.7% roughly 2 in 3 raw findings were false alarms before triage

Result — the multi-turn loop pays off

VulnHunterX

High-confidence accuracy, by how many turns the LLM took (OWASP-Python · DeepSeek).

81.8%

stop after 1 turn

commit too early, miss the defense



96.9%

after a 2nd turn

expand context, then decide

High-confidence accuracy 94.2% well-calibrated over 275 high-confidence verdicts

Mean 2.6 turns the loop expands context only when it needs to (max 5)

\$0.40 for 300 findings on DeepSeek; \$0.00 with a local Ollama model

Result — per-CWE and across models

VulnHunterX

CWE-89 · SQLi

F1 100%

CWE-79 · XSS

F1 100%

**CWE-330 · weak
RNG**

F1 100%

CWE-643 · XPath

F1 84.4%

CWE-611 · XXE

F1 50% — hard case

Same approach, three models *OWASP-Java · VulnHunterX*

DeepSeek F1 92.0% · FP-reduction 78.6% · \$0.30

GPT-4.1-mini F1 85.7% · FP-reduction 58.1% · \$0.80

Ollama Qwen (local) FP-reduction 100% · \$0.00 — runs entirely offline

Focused slice: DeepSeek, June 2026 runs — directional, not a universal claim.

 PART 2

How to install & use

02



The four tools — what each does

VulnHunterX

CodeQL

Semantic, database-driven taint analysis — the deepest engine. Needs a DB build.

core

Semgrep

Fast pattern-based rules across many languages. No DB needed.

breadth

OpenGrep

Semgrep-compatible fork — same rules, drop-in alternative.

alt

tree-sitter

Syntactic context extraction — pip-installed fallback when there's no CodeQL DB.

fallback



```
# Prerequisites: Python 3.12+, CodeQL CLI 2.15+, an LLM provider key  
git clone https://github.com/vinsoc-cyber/VulnHunterX.git && cd VulnHunterX  
  
uv venv --python python3.12 .venv && source .venv/bin/activate  
uv pip install -e ".[dev]" # plain python3.12 -m venv + pip also works  
  
cp env.example .env # add OPENAI / ANTHROPIC / OLLAMA keys  
vuln-hunter-x check-env # verify the whole toolchain is green
```

Configuration you need to set

```
                # .env - provider keys + optional tool paths
LLM_PROVIDER=openai          # openai | anthropic | ollama
LLM_MODEL=gpt-4o
OPENAI_API_KEY=sk-...        # or ANTHROPIC_API_KEY / OLLAMA_API_BASE
CODEQL_PATH=/path/to/codeql  # only if codeql isn't on your PATH
SEMGREP_PATH=/path/to/semgrep # only if semgrep isn't on your PATH

# config/confirm_findings.yaml - verification tuning
provider: openai  model: gpt-4o  temperature: 0.2  max_iterations: 3

# config/repos.yaml - batch targets (name, url, language, build_command)
```

Three ways to start a scan



1) Friendliest: a guided wizard (prompts for everything, live-tests your LLM)

```
vuln-hunter-x interactive
```

2) One-shot: the whole pipeline in a single command

```
vuln-hunter-x scan --url https://github.com/org/app.git \  
                  --lang python --profile extended --limit 5
```

3) Stage by stage (full control)

```
vuln-hunter-x prepare --repo pyyaml  
vuln-hunter-x analyze --repo pyyaml --profile extended  
vuln-hunter-x verify  --repo pyyaml --limit 5  
vuln-hunter-x report  --repo pyyaml --lang python
```

 PART 3

Real example — dvpwa



dvpwa — a deliberately vulnerable web app

VulnHunterX

An intentionally insecure Python **aiohttp** app (github.com/anxolerd/dvpwa) — scanned next to a clean library.

dvpwa *vulnerable*

SQL injection · stored XSS · weak MD5 ·
missing authorization · CSRF disabled

→ **VulnHunterX should surface real TPs**

pyyaml *real-world*

a mature, widely-used YAML library
with no planted vulnerabilities

→ **false positives should be suppressed**

The value proposition in one screen:

the vulnerable app lights up with real bugs; the clean library stays quiet.

Ground truth (realvuln): 22 real vulnerabilities + 4 false-positive traps.

Run the dvpwa example



```
# one command: clone + analyze + verify + report, for both repos
python examples/pipeline_python.py # add --dry-run to preview

# or drive just dvpwa, stage by stage:
vuln-hunter-x prepare --repo dvpwa # clone + CodeQL DB + context
vuln-hunter-x analyze --repo dvpwa --profile extended
vuln-hunter-x verify --repo dvpwa --limit 5 --report

# read the verdicts the LLM produced:
cat output/python/dvpwa/verification_results/report.md
```

dvpwa answer key — verify your results

VulnHunterX

SQL injection · CWE-89

dao/student.py:42 — query built with Python % then execute(q)

TP

Stored XSS ×5 · CWE-79

templates render unescaped — root cause autoescape=False (app.py:35)

TP

Weak password hash · CWE-916

dao/user.py:41 — unsalted MD5

TP

Missing authz / CSRF off

views.py (CWE-862) · csrf middleware commented out, app.py:27 (CWE-352)

TP

NOT SQL injection

dao/user.py · review.py · course.py — parameterized %s / % (name)s queries

trap

NOT XSS

student.jinja2:14 — {{ name | e }} is explicitly escaped

trap

Ground truth: 22 real vulnerabilities + 4 false-positive traps

Score yourself: flagging student.py SQLi = correct; flagging the parameterized DAO queries or the escaped template = a false positive your run should suppress.

 PART 4

Homework — dvcp

04



Homework — scan dvcp

Your task: run it, open the report, list the TPs and FPs — then check against the answer key.

```
# config/repos.yaml already defines dvcp (C needs a build command):  
# url: github.com/hardik05/Damn_Vulnerable_C_Program  
# build_command: gcc -g -o dvcp imgRead.c  
  
# run the C pipeline – contrast target is c-ares (a real DNS library):  
python examples/pipeline_c.py # add --dry-run to preview  
  
# or drive just dvcp, stage by stage:  
vuln-hunter-x prepare --repo dvcp # clone + build + CodeQL DB  
vuln-hunter-x analyze --repo dvcp  
vuln-hunter-x verify --repo dvcp --limit 5 --report
```

dvcp answer key — what to expect

VulnHunterX

Integer overflow · CWE-190

width + height, width * height

TP

Integer underflow · CWE-191

width - height + 100

TP

Heap buffer overflow · CWE-122

memcpy into a buffer sized from the overflowed value

TP

Double-free · CWE-415

free(buff1) again when size1 % 2 == 0

TP

Use-after-free · CWE-416

buff1[0]='a' after free when size1 % 3 == 0

TP

OOB read / write · CWE-125/787

buff3[size3], buff4[size3] past the bounds

TP

Divide-by-zero · CWE-369

width / height when height == 0

TP

Leak + infinite recursion

CWE-401: buff4=0 without free · CWE-674: stack_operation, recurses forever

TP

All bugs live in ProcessImage() — reachable from attacker-controlled image-header fields (width / height / data).

Contrast: c-ares (production DNS library) should come back mostly false positives.

Answer key derived from the upstream imgRead.c source (external repo).

Recap & references

What it is: SAST finds candidates; a multi-turn LLM verifies them with evidence

Results: OWASP-Python: precision 37.7% → 87.3%, 91.4% of false positives removed

Install & use: prepare → analyze → verify → report · vuln-hunter-x interactive

Practice: dvpwa example today · dvcp homework — check both against the answer keys

Start here → [vuln-hunter-x interactive](#)

References: Vulnhalla (CyberArk) · CodeQL docs · Semgrep docs · SARIF spec · repo README + config/RULES.md